

AWS Practice Task's

Task 1: Deploy a Simple Web Page on an EC2 Instance

1. **Launch an EC2 Instance:**
 - **Custom VPC Setup:**
Launch an EC2 instance in a **public subnet** of your custom VPC or the default VPC.
 - **Security Group Configuration:**
Configure a **Security Group** that allows the following:
 - **SSH (port 22):** Allow traffic from your IP address.
 - **HTTP (port 80):** Allow traffic from anywhere (0.0.0.0/0).
2. **Configure User Data Script for Web Server:**
 - Use the **user data script** to automate the installation of the Apache HTTP server and configure it to serve a simple web page.
 - This script ensures that the Apache server starts and the simple HTML page is served on port 80.
3. **Launch and Connect to the EC2 Instance:**
 - Launch the EC2 instance and ensure it is assigned a **public IP**. Use the SSH key pair to connect to the instance.
4. **Verify the Web Page:**
 - Open a browser and navigate to the **public IP address** of your EC2 instance.
 - You should see the simple **"Hello, World!"** web page.

Task 2: Deploy a Flask Application on an EC2 Instance

1. **Launch an EC2 Instance:**
 - **Custom VPC Setup:**
Launch an EC2 instance in a public subnet within your custom VPC or the default VPC.
 - **Security Group Configuration:**
Create a Security Group with the following inbound rules:
 - **SSH (port 22):** Allow access from your IP address.
 - **HTTP (port 80):** Allow access from anywhere (0.0.0.0/0).
2. **Install Flask on EC2 Instance:**
 - SSH into the EC2 instance after it is launched.
 - Install Python 3 and Flask by running the following commands.
3. **Write a Python(Flask app) file called `app.py`**
 - This app will respond with "Your name" when accessed.
4. **Run the Flask Application:**
 - The application will be accessible on the instance's public IP on port 80.
5. **Verify the Deployment:**
 - Open a web browser and access the public IP address of the EC2 instance.
 - You should see the message "Your name" displayed in the browser, confirming the Flask app is running successfully.

AWS Practice Task's

Task 3: Add Routes to the Flask Application

1. **Complete Task 1:**
 - Follow **Task 1** to deploy a basic Flask application that returns "Your name" when accessed.
2. **Add Additional Routes to the Flask Application:**
 - Modify the `app.py` file to include the following routes:
 - **Homepage (/):** Displays a welcome message.
 - **About Page (/about):** Displays information about the application.
 - **Contact Page (/contact):** Provides contact information.
 - **Status Page (/status):** Displays the current status of the app.
3. **Run the Flask Application:**
 - Restart the Flask application to apply the changes.
4. **Verify Deployment:**
 - Open a web browser and access the following routes using the **public IP** of the EC2 instance.
 - Verify that the content of each route is displayed correctly.

Task 4: Set Up a Static Website Using Amazon S3 to Display Your Portfolio

1. **Create an S3 Bucket:**
 - In the S3 Console, click on Create bucket.
 - Enter a globally unique name for your bucket (e.g., `my-portfolio-site`).
 - Select a region closest to your target audience.
 - Leave the other settings at their default values and click Create.
2. **Enable Static Website Hosting:**
 - After the bucket is created, go to the Properties tab.
 - Scroll down to the Static website hosting section and click Edit.
 - Enable Static website hosting and select Use this bucket to host a website.
 - Set the Index document to `index.html` (or the name of your homepage).
 - Optionally, you can specify an Error document (e.g., `error.html`).
3. **Upload Your Portfolio HTML Files:**
 - In the S3 Console, navigate to your bucket and click Upload.
 - Add your `index.html` file and any other supporting files (CSS, images, etc.) to the bucket.
 - Set the appropriate permissions to allow public access to these files:
 - Ensure your files are publicly accessible by setting the ACL (Access Control List) for the objects to Public.
 - Alternatively, configure a Bucket Policy to allow public access.
4. **Configure Bucket Permissions for Public Access:**
 - Go to the Permissions tab of the bucket and under Block public access (bucket settings), ensure public access is allowed for the objects.
 - Apply the necessary bucket policy to allow access to the files for everyone.

AWS Practice Task's

5. Test Your Website:

- After the website is set up, go to the Static website hosting section of the S3 bucket and copy the S3 website endpoint URL.
- Open a web browser and paste the URL to access your portfolio site.

Task 5: Deploy a Dockerized Web Application on EC2 Instances

1. Launch Two EC2 Instances:

- Create a VPC with at least one public subnet (or use the default VPC).
- Launch two EC2 instances in the public subnet, ensuring each instance gets a public IP for external access.

2. Configure Security Groups:

- Create a Security Group for the EC2 instances with the following rules:
 - **SSH (port 22):** Allow access from your IP address.
 - **HTTP (port 80):** Allow access from anywhere (0.0.0.0/0).

3. Automate Docker Installation:

- During EC2 instance creation, attach the following **user data script** to automate

4. Deploy a Dockerized Application:

- SSH into both EC2 instances and pull a sample Docker image
- Run the Docker container on port 80 using the host network:

5. Verify the Deployment:

- Open your browser and access the **public IP address** of each EC2 instance.
- Confirm the Dockerized application is running and accessible.

Task 6 : Create and Use a Custom AMI with Docker Pre-installed

1. Prepare the EC2 Instance:

- Choose one of the EC2 instances configured in Task 5.
- Ensure Docker is installed and running. Create or update the **user data script** on this instance to automatically start Docker on boot.

2. Create a Custom AMI:

- Stop the selected EC2 instance.
- Go to the EC2 dashboard, select the instance, and create an AMI:
 - Provide a name and description for the AMI.
 - Include all configurations (like Docker installation and user data).

3. Launch a New EC2 Instance Using the Custom AMI:

- Use the AMI created to launch a new EC2 instance.
- Ensure the new instance is assigned a public IP and has the same Security Group as in Task 5.

4. Verify Docker Pre-installation:

- SSH into the new instance and run:
- Verify Docker is pre-installed, running, and ready to use.

AWS Practice Task's

Task 7: Set Up a Load Balancer for Dockerized Applications

1. **Create an Application Load Balancer (ALB):**
 - Navigate to the **EC2 Dashboard > Load Balancers**.
 - Create an ALB:
 - Type: **Application Load Balancer**.
 - Scheme: **Internet-facing**.
 - Subnets: Choose at least two subnets in different Availability Zones.
 - Security Group: Create or use a Security Group with:
 - **HTTP (port 80)**: Allow traffic from anywhere (0.0.0.0/0).
2. **Create a Target Group:**
 - Navigate to the **Target Groups** section in the EC2 Dashboard.
 - Create a new Target Group:
 - Target type: **Instances**.
 - Protocol: **HTTP** (port 80).
 - Health Check Path: **/**.
3. **Register Instances with the Target Group:**
 - Add the two EC2 instances created in Task 5 to the Target Group.
4. **Attach the Target Group to the Load Balancer:**
 - Edit the ALB's listener rules to forward HTTP traffic to the Target Group.
5. **Verify the Load Balancer:**
 - Obtain the DNS name of the ALB from the Load Balancer settings.
 - Access the DNS name in your browser.
 - Verify that the traffic is being distributed between the two EC2 instances.

Task 8: Deploy a Dockerized Node.js Application Using Amazon ECR and ECS

1. **Create a Docker Image for the Application:**
 - Create a simple **Node.js** or **Flask** application (e.g., a Travel Memory app).
 - Create a **Dockerfile** for the application to build a Docker image.
 - Build the Docker image
2. **Push the Docker Image to Amazon ECR:**
 - In the **ECR Console**, create a new **repository** (e.g., **travel-memory-app**).
 - Authenticate Docker to your ECR repository.
 - Tag the image for ECR.
 - Push the Docker image to your ECR repository:
3. **Create an ECS Cluster:**
 - Go to the **ECS Console** and create a new **ECS cluster** using **Fargate**.
 - Select the cluster template as **Networking only** (Fargate).
4. **Create a Task Definition:**
 - In the **ECS Console**, create a new **Task Definition**:
 - Choose the **Fargate** launch type.
 - Under **Container definitions**, add a container with the following settings:

AWS Practice Task's

- Image
 - Port mapping: 8080 (or the port your app listens on).
 - Set the memory and CPU limits for the container.
5. **Create a Load Balancer:**
 - In the **EC2 Console**, create an **Application Load Balancer**.
 - Set it to route traffic to the ECS task via the target group.
 - Add a listener for HTTP (port 80) and ensure it routes traffic to the target group associated with your ECS tasks.
 6. **Deploy the Application Using ECS Fargate:**
 - In the **ECS Console**, create a new **Service** for your ECS cluster.
 - Select **Fargate** as the launch type and choose the task definition created earlier.
 - Set the service to use the **Application Load Balancer** for traffic distribution.
 - Define the number of tasks (containers) to run.
 7. **Access the Application:**
 - After the ECS service is deployed, go to the **Load Balancer** section in the EC2 Console.
 - Copy the **DNS name** of the Load Balancer and open it in a web browser.
 - Verify that the **Node.js or Flask application** is running and accessible via the Load Balancer's DNS.

Please refer to this [link](#) for the necessary inputs to complete this task.

From Wednesday onwards, the solution file for this task will be available at this [link](#)